

Altair installation with Conda can be carried out with the following command:

```
conda install altair --channel conda-forge
```

Altair has a very simple-to-use API. As the official documentation highlights, Altair enables the developer to build elegant visualisation with very minimal code.

```
from altair import Chart, load_dataset
```

```
# load data as a pandas DataFrame
cars = load_dataset('cars')
```

```
Chart(cars).mark_point().encode(
    x='Horsepower',
    y='Miles_per_Gallon',
    color='Origin',
```

The output of the above code is shown in Figure 4. The data source is DataFrame.

The major benefit of using Altair is that the developer has to specify only what is required and does not have to specify how to produce it. Altair has features to render various types of charts. For example, a code-snippet to produce a Streamgraph

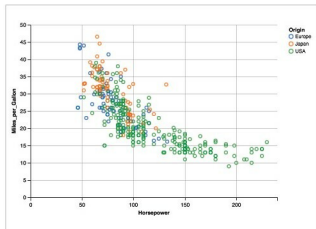


Figure 4: Altair sample visualisation

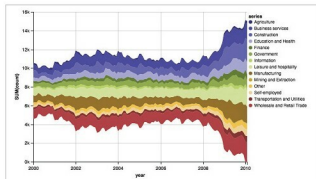


Figure 5: Altair Streamgraph

is shown below and its output is shown in Figure 5.

```
Chart(data).mark_area(stacked='center').encode(
    X('date:T', timeUnit='yearmonth',
      axis=Axis(format='%Y', labelAngle=0, title='year')),
    Y('sum(count)'),
    color=Color('series:N', scale=Scale(range='category20b'))
)
```

A tutorial on Altair is available at <https://github.com/altair-viz/altair/blob/273a1fc9cec1956474a755d5fe32f0e3f0aee8/altair/notebooks/02-Introduction.ipynb>.

Seaborn

The Seaborn library is aimed at building informative statistical graphics in Python. It is built on top of Matplotlib. Seaborn also has tight integration with the PyData stack. The major features of the Seaborn library are:

- The availability of various built-in themes, which help in improving the aesthetics of the visualisation.
 - The support for various statistical features such as linear regression models and functions to plot statistical time series.
- Seaborn can be installed easily with the following command:

```
pip install seaborn
```

It provides options to build a variety of graphs. The Hexbin plot-building source code is shown below:

```
x, y = np.random.multivariate_normal(mean, cov,
                                     1000).T
with sns.axes_style("white"):
    sns.jointplot(x=x, y=y, kind="hex",
                 color="k");
```

The output of the above code is shown in Figure 6.

A Violin plot made with Seaborn is shown in Figure 7. The source code is shown below:

```
import seaborn as sns
import matplotlib.pyplot as plt
sns.set(style='whitegrid')

# Load the example dataset of brain network correlations
df = sns.load_dataset("brain_networks", header=[0, 1, 2],
                    index_col=0)

# Pull out a specific subset of networks
used_networks = [1, 3, 4, 5, 6, 7, 8, 11, 12, 13, 16, 17]
used_columns = (df.columns.get_level_values("network")
                .astype(int)
                .isin(used_networks))

df = df.loc[:, used_columns]
```